StateShield-MQTT: First Review Brief

Faculty Project 2: MQTT Protocol Security | Faculty: Krishna Kumari | Review: 23 October 2026
B4: Vinay Kumar (AP26122050024), Triphal Sahu (AP26122050013), Venkatesh (AP26122050014)

Question and boundary

MQTT 5.0 can retain subscriptions and QoS 1 messages for a disconnected client until its Session Expiry Interval ends. This helps devices recover after a network outage. It also creates broker state through a sequence of valid actions by different clients. We ask whether a state-aware runtime policy can limit harmful accumulation while preserving legitimate offline delivery. This first review measures the mechanism on one broker. It does not claim a vulnerability or a completed defense.

Controlled sequence

Three subscribers connect to a local broker, subscribe to one topic at QoS 1, and disconnect. A separate publisher sends five small messages. We reconnect the same subscriber Client IDs and count recovered messages. The clean control sets Session Expiry to 0; the durable case sets it to 120 seconds. We compare Mosquitto's existing max_queued_messages setting of 1000 with a cap of 3. All traffic stays on 127.0.0.1:18883. Python 3.12.10, Paho 2.1.0, and Mosquitto 2.1.2 were used.

State transitions expected from the standard

With expiry 0, a disconnected client leaves no resumable session. With expiry 120 seconds, its subscription remains available during the offline interval. On reconnect, the broker's Session Present flag tells us whether it resumed state. The publisher is a separate client, so the observed queue is a cross-client effect. All CONNECT, SUBSCRIBE, PUBLISH, and DISCONNECT actions in this pilot are individually valid MQTT 5.0 actions.

Actual local results (18 October 2026)

Scenario	Queue cap	Recovered / subscriber	Store count
Online delivery, both runs	1000 / 3	1 of 1	Not a queue test
Clean session, expiry 0	1000 / 3	0 of 5	55 to 55
Durable session, expiry 120 s	1000	5 of 5	56 to 61
Durable session, expiry 120 s	3	3 of 5	56 to 59

The broker store counter is not the sum of deliveries to three clients. With the three-message queue cap, each offline client recovered only three messages. This is a trade-off from an existing broker control. The observed retention is permitted, configuration-dependent MQTT 5.0 behavior.

Docker memory readings were about 2.8-3.1 MiB at sampled moments. They are too noisy and few to support a performance conclusion.

Interpretation and limits of this measurement

The three durable subscriptions remained visible in the broker's disconnected-client and subscription counters while offline. Store-message count rose from 56 to 61 in the uncapped run and from 56 to 59 in the capped run. These are broker-level aggregates, and the base value includes broker-managed messages. We therefore use within-run changes and per-client reconnection results, not the absolute store count, as evidence. One short run cannot estimate an attack threshold or prove memory exhaustion.

Related Work and Proposed Direction

Where this sits in the literature

Song et al.'s MBFuzzer (USENIX Security 2025) is the closest work: it uses multi-party MQTT fuzzing across six brokers and reports 73 bugs. Multi-client testing is therefore not our novelty. ProtocolGuard (NDSS 2026) targets protocol non-compliance using static and dynamic analysis. MQTT-I (IEEE TDSC 2024) addresses end-to-end integrity with an untrusted broker. SIMMS (IEEE TETC 2026) studies broader multi-protocol gateway security. Our possible question is different: runtime handling of spec-permitted persistent-state pressure. We still need to establish whether it adds value beyond native broker limits.

Three candidate invariants

1. Per-session queue budget: bound offline queued messages or bytes. Risk: a genuine outage loses data.

2. Global retained-state budget: watch disconnected sessions, queue age, and memory over time. Risk: a fleet restart resembles abuse.

3. Durable-session creation rate: limit repeatedly abandoned sessions per authenticated identity. Risk: legitimate device replacement or shared credentials.

Next implementation and evaluation

Instrument per-client broker state or use broker hooks, then build one conservative state-aware intervention. Compare it with passive monitoring and Mosquitto's queue limit. Repeat tests under benign outages and controlled accumulation, measuring prevented state growth, false interventions, legitimate delivery, peak CPU/memory, and latency. Only after the first-broker result is repeatable should we test another broker. The present pilot has one run per configuration and no implemented intervention.

Feasibility and decision criteria

The environment is available now: Docker Mosquitto, Paho Python clients, broker $SYS counters, and saved JSON results. The missing piece is per-session queue visibility or enforcement through broker integration. We will create a benign outage dataset alongside controlled accumulation, repeat each load, and report both successful mitigation and its delivery cost. A useful policy must beat a fixed queue cap on at least one stated trade-off without an unacceptable false-intervention rate. If it cannot, the honest result is that native limits are adequate for this case.

Decision requested at first review

Is persistent-session pressure an appropriate first MQTT interaction category, or should the group switch to a clearer multi-client semantic case? Faculty feedback will determine the next experiment. CLA-3 individual topics still need member approval and sheet registration before 22 October, 11:59 PM.

Sources

OASIS MQTT 5.0: https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html

Mosquitto configuration and $SYS manuals: https://mosquitto.org/man/mosquitto-conf-5.html ; https://mosquitto.org/man/mosquitto-8.html

Song et al., MBFuzzer, USENIX Security 2025: https://www.usenix.org/conference/usenixsecurity25/presentation/song-xiangpu

Song et al., ProtocolGuard, NDSS 2026: https://www.ndss-symposium.org/wp-content/uploads/2026-f521-paper.pdf

Buccafurri et al., MQTT-I, IEEE TDSC 2024: https://iris.unirc.it/handle/20.500.12318/142287

SIMMS, IEEE TETC 2026: https://xplorestaging.ieee.org/document/11480700/